

CompuFix Agents

Informe Técnico del Proyecto

Sistema multiagente para diagnóstico y asistencia
en problemas comunes de uso de computadoras

Materia: Procesamiento de Lenguaje Natural III
Institución: Universidad de Buenos Aires (UBA)

Docentes:

- Oksana Bokhonok
- Abraham Rodriguez

Integrantes del equipo:

1. Emmanuel Cardozo — Workflow + Memoria conversacional
2. Carlos Villalobos — Informe técnico + RAG
3. Lucas Didone — Sistema de agentes + Tools + UI

Fecha: 10/06/2026

1. Objetivo del proyecto

CompuFix Agents es un sistema multiagente (MVP) desarrollado como trabajo final de la materia Procesamiento de Lenguaje Natural III. Su objetivo es diagnosticar y asistir al usuario ante tres clases de problemas frecuentes en el uso diario de una computadora:

- Bibliotecas de Python faltantes — detecta errores como `ModuleNotFoundError`, mapea el nombre de importación al paquete pip correcto (p. ej. `cv2` → `opencv-python`), propone instalación y verifica la importación.
- Lentitud de red — analiza un estado de red simulado y recomienda cambiar de una red Wi-Fi 2.4 GHz lenta a una 5 GHz más rápida (mock).
- Alto consumo de recursos — lista los procesos con mayor uso de CPU/RAM mediante `psutil` y, con aprobación explícita, puede terminar uno (dry-run por defecto).

El diseño prioriza la seguridad: las acciones sensibles requieren aprobación humana explícita y varias están simuladas. El sistema puede operar de forma completamente local y determinística sin clave de API de LLM.

2. Arquitectura general

2.1 Diagrama de flujo

```
user input
-> Triage Agent      (classify problem + extract entities)
-> Diagnostic Agent  (RAG over local knowledge base)
-> Planner & Safety  (build safe action plan, mark approvals)
-> Approval check    (human approves sensitive steps)
-> Executor Agent    (runs ONLY known, controlled tools)
-> final response
```

Más detalles en el siguiente documento: [architecture.md](#).

2.2 Orquestación

La orquestación se implementa con un grafo `LangGraph` (`triage` → `diagnostic` → `planner` → `executor`) con `MemorySaver` e `interrupt_before` en el executor para `human-in-the-loop`, y `helpers` determinísticos usados por la interfaz `Streamlit`.

2.3 Componentes principales

Componente	Ubicación	Función
Triage Agent	agents/triage_agent.py	Clasificación híbrida (reglas + LLM opcional)
Diagnostic Agent	agents/diagnostic_agent.py	Diagnóstico fundamentado en RAG
Planner Agent	agents/planner_agent.py	Plantillas de plan + <code>enforce_safety</code>
Executor Agent	agents/executor_agent.py	Allowlist, aprobación y ejecución

RAG	rag/	Ingesta, Chroma, retriever léxico/vectorial
Tools	tools/	8 herramientas controladas (Python, red, procesos)
UI	app/streamlit_app.py	Análisis, aprobación y ejecución interactiva
Evaluación	eval/run_eval.py	Métricas sobre casos etiquetados

Más detalles en el siguiente documento: [architecture.md](#).

3. Implementación técnica

3.1 Stack tecnológico

Categoría	Tecnología	Versión / notas
Lenguaje	Python	3.11–3.12 (numpy < 2.0)
Orquestación	LangGraph + LangChain	0.2.x / 0.3.x
Modelos LLM	OpenAI (opcional)	Solo con OPENAI_API_KEY
Embeddings	OpenAI / sentence-transformers	auto openai local none
Vector store	ChromaDB + langchain-chroma	Persistido en data/vectorstore
UI	Streamlit	≥ 1.36
Sistema	psutil	Listado y terminación de procesos
Validación	Pydantic v2	Esquemas tipados
Tests	pytest + ruff	82 tests automatizados

3.2 Herramientas controladas (allowlist)

Herramienta	Efectos	Aprobación
check_python_package	Solo lectura	No
install_python_package	pip install en intérprete activo	Sí
verify_python_import	Solo lectura (subprocess)	No
get_current_network	Lectura (mock)	No
list_available_networks	Lectura (mock)	No
switch_network	Mutación en memoria (simulado)	Sí
list_top_processes	Lectura real (psutil)	No
kill_process	Dry-run por defecto; gated	Sí

3.3 Base de conocimiento RAG

La base de conocimiento contiene 8 documentos Markdown (python/, network/, system/), divididos en 21 chunks. El retriever usa Chroma + embeddings o cae a un scorer léxico con stopwords EN/ES.

3.4 Política de seguridad

- Principio de mínimo privilegio: el executor solo invoca funciones del registro.
- Fail-safe: herramientas desconocidas se tratan como sensibles y se omiten.
- Sin ejecución de shell arbitrario ni borrado de archivos.
- Procesos críticos protegidos (launchd, systemd, WindowServer, etc.).
- ALLOW_REAL_PROCESS_KILL=false por defecto.

4. Evaluación

Se ejecutó el harness eval/run_eval.py sobre 7 casos etiquetados (use_llm=False) y la suite pytest (82 tests) sobre agentes, RAG, tools y workflow.

4.1 Métricas del pipeline (triage + planner)

Métrica	Resultado	Descripción
Casos evaluados	7	Entradas en test_cases.json
Precisión de triage	100%	problem_type predicho == esperado
Cobertura de herramientas	100%	Herramienta esperada en el plan
Precisión de aprobación	100%	Flag requires_approval correcto
Mapeo de paquetes	100%	cv2→opencv-python, sklearn→scikit-learn, yaml→PyYAML

4.2 Métricas de agentes y RAG (tests unitarios)

Área	Tests	Qué valida
Triage Agent	7	Regex de módulos, keywords EN/ES, entidades
Diagnostic Agent	4	Diagnóstico fundamentado, docs recuperados
Planner Agent	8	Plantillas por tipo, enforce_safety
Executor Agent	6	Allowlist, aprobación, skips
RAG / Embeddings	7	Backends, fallback léxico
Python env tools	5	Mapeo import→pip, install/verify
Network tools	6	Estado mock, switch simulado
Process tools	6	Top procesos, dry-run, protegidos
Workflow E2E	7	Pipeline completo LangGraph + helpers
Total pytest	82	100% passed (exit code 0)

5. Resultados y ejemplos

5.1 Caso 1 — Biblioteca Python faltante

Entrada: `ModuleNotFoundError: No module named 'cv2'`

- Triage: `python_missing_library` (confianza ~0.95)
- Entidades: `missing_module=cv2`, `package_name=opencv-python`
- Plan: `check` → `install` (aprobación) → `verify import`

5.2 Caso 2 — Red lenta

Entrada: `Mi internet está muy lento`

- Triage: `network_slow`
- Plan: `get_current_network` → `list_available` → `switch` (aprobación, simulado)

5.3 Caso 3 — Alto uso de recursos

Entrada: `La computadora está lenta y consume mucha RAM`

- Triage: `high_resource_usage`
- Plan: `list_top_processes(limit=5)` — procesos reales vía `psutil`

6. Conclusiones y mejoras futuras

6.1 Conclusiones

- MVP funcional, auditable y seguro con 100% de precisión en el conjunto de evaluación.
- Arquitectura multiagente con human-in-the-loop aplicable a asistentes del SO.
- Fallback léxico del RAG garantiza operación offline.
- 82 tests y harness de evaluación para regresión continua.

6.2 Mejoras futuras

- Inspección y cambio de red real (opt-in, sandboxed) por SO.
- Embeddings locales por defecto y base de conocimiento ampliada.
- Planificación asistida por LLM restringida al allowlist.
- Nuevas categorías: espacio en disco, drivers, DNS, etc.
- Audit log persistente y cola de aprobaciones en LangGraph.

7. Planificación del equipo

7.1 Reparto de tareas para 3 estudiantes

El proyecto fue desarrollado en equipo por tres integrantes, con responsabilidades claramente delimitadas según los módulos del repositorio.

Estudiante	Responsabilidades	Entregables
Emmanuel Cardozo (1 — Workflow + Memoria conversacional)	Definir memoria compartida y workflow de orquestación.	graph/ workflow.py , agents/
Carlos Villalobos (2 —Informe técnico + RAG)	Crear documentos, ingesta, vector DB, retrieval, prompts del diagnosticador y dataset de prueba.	rag/, reports/
Lucas Didone (3 — Sistema de agentes + Tools + UI)	Implementar sistema de agentes, tools, Streamlit App, aprobación humana (human-in-the-loop), logs y demo.	agents/, tools/, app/streamlit_app.py, eval/